EduStack Masterclass

Your Ultimate Computer Science & Engineering Hub

VOLUME 3 — Database Engineering, Data Modeling & Cloud Services

Tier-1 Interview Reference Guide (Visa, Amazon, Oracle, JPMC, Microsoft)

MongoDB Atlas | Mongoose Schemas | TTL Indexes | Multer MemoryStorage | 25 Tier-1 Q&As

Developer:	Shubham Kumar | CSE Student | NIT Patna
Target Roles:	Database Architect, SDE II Backend, Cloud Infrastructure Lead
Database:	MongoDB Atlas M0 + Mongoose ODM (Schema Validation, Indexes, TTL)
Schemas:	User (select:false), OTP (TTL), Subject, Resource, Payment, Notification
Cloud Ops:	Multer memoryStorage + Cloudinary CDN + Nodemailer SMTP
Data Sync:	Google Sheet Live Sync via CSV with 3-Layer In-Memory Caching
Volume 3:	MongoDB Design, Indexes, Cloud File Pipelines & 25 Q&As

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub

For SDE Technical Interview Preparation — Volume 3 of 4

Table of Contents — Volume 3

SECTION 13

MongoDB Architecture & Mongoose ODM Internals

NoSQL vs RDBMS evaluation for tier-1 tech interviews

13.1 NoSQL Document Store vs Relational RDBMS

In enterprise interviews at Amazon, Oracle, and Microsoft, database selection must be justified based on query patterns, schema flexibility, and horizontal scalability. EduStack uses MongoDB Atlas - a distributed BSON document database.

Dimension	MongoDB (EduStack)	PostgreSQL / Oracle
Schema Model	Dynamic BSON documents - handles varying resource fields naturally	Rigid relational tables - requires ALTER TABLE migrations
Query Model	JSON-native Mongoose queries & Aggregation Pipelines	SQL JOIN queries with relational algebra optimization
Scale Model	Native sharding across cluster nodes	Vertical scaling (primary-replica read scaling)
Joins / Population	populate() batched $in lookups or $lookup pipeline	Native C-level JOIN execution
Special Features	Built-in TTL (Time-To-Live) index engine for auto-deletions	Requires pg_cron or external background workers

SECTION 14

Complete Schema Blueprint & Implementation

Production Mongoose schemas for User, OTP, Subject, Resource, Payment, Notification

Mongoose Schema & Database Implementation

```
// models/otp.js  -  MongoDB TTL Index Engine
const mongoose = require('mongoose');

const otpSchema = new mongoose.Schema({
  email:     { type: String, required: true, lowercase: true, index: true },
  hashedOtp: { type: String, required: true }, // bcrypt hash of 6-digit code
  purpose:   { type: String, enum: ['verify', 'reset'], required: true },
  used:      { type: Boolean, default: false },
  expiresAt: { type: Date, required: true },
});

// TTL INDEX: MongoDB background thread auto-deletes doc when expiresAt passes
otpSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });

module.exports = mongoose.model('OTP', otpSchema);
```

SECTION 17

Cloud Infrastructure & Zero-Disk Upload Pipeline

Multer memoryStorage RAM buffers directly streamed to Cloudinary CDN

Render.com operates on ephemeral storage - files saved to server disk are erased on redeploy. EduStack uses Multer memoryStorage to capture file bytes into RAM Buffer, then streams directly to Cloudinary CDN via upload_stream().

Mongoose Schema & Database Implementation

```
// Zero-Disk Upload Pipeline
const multer = require('multer');
const cloudinary = require('cloudinary').v2;

const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 5*1024*1024 } });

const streamToCloudinary = (buffer, folder) =>
  new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream({ folder }, (err, res) => {
      if (err) return reject(err);
      resolve(res);
    });
    stream.end(buffer);
  });
```

SECTION 19

Tier-1 Interview Q&A - Database & Cloud

25 Deep Technical Questions asked by Visa, Amazon, Oracle, JPMC, Microsoft

Q: Explain the N+1 query problem in Mongoose and how populate() resolves it.

Comprehensive Database Answer:

The N+1 query problem occurs when fetching 1 parent document and then executing N individual queries to fetch child details. Mongoose populate() solves this by executing 2 queries total: Query 1 finds parent documents; Query 2 executes child.find({ _id: { $in: childIds } }), batching all child IDs in a single $in query.

-> Parent query: Subject.find() -> returns 50 subjects.
-> Populate query: Resource.find({ _id: { $in: [50 IDs] } }) -> 2 queries total.
-> Alternative: MongoDB $lookup aggregation pipeline executes in a single query pass.

Q: How does MongoDB's TTL index background engine work?

Comprehensive Database Answer:

A TTL (Time-To-Live) index is created on a Date field. MongoDB runs a background cleanup thread once every 60 seconds that evaluates current_time > indexed_date + expireAfterSeconds and removes matching documents from memory and disk B-tree indexes.

-> Zero application code or cron jobs required.
-> Used in EduStack for automatic 5-minute OTP expiration.